

19. Mediator Design Pattern in Java

19.1 Introduction

The **Mediator Design Pattern** is a **behavioral pattern** that reduces coupling between components by having them communicate through a mediator rather than directly.

19.2 Problem it Solves

When multiple objects interact in complex ways, this pattern centralizes their communication, reducing dependencies.

19.3 What is Mediator Pattern?

It defines a language's grammar and provides an interpreter to interpret sentences written in the language.

19.4 Real-World Analogy

A **chatroom** where users send messages to the room and the room delivers them to the correct recipient.

19.5 Structure

```
package com.mannemlokesh;

public interface Mediator {
    void send(String message, Colleague colleague);
}

abstract class Colleague {
    protected Mediator mediator;

    public Colleague(Mediator mediator) {
        this.mediator = mediator;
    }
}
```

19.6 Benefits

- Promotes loose coupling
- Centralizes complex communications
- Improves code maintainability

19.7 Implementation Steps

1. Define a Mediator interface
2. Implement concrete mediator
3. Define Colleague class
4. Concrete colleagues use mediator to communicate

19.8 Examples

Example1: Chatroom Mediator

```
package com.mannemlokesh.example1;
```

```
import java.util.ArrayList;
import java.util.List;

interface ChatMediator {
    void sendMessage(String message, User user);

    void addUser(User user);
}

class ChatRoom implements ChatMediator {
    private List<User> users = new ArrayList<>();

    public void addUser(User user) {
        users.add(user);
    }

    public void sendMessage(String message, User sender) {
        for (User user : users) {
            if (user != sender) {
                user.receive(message);
            }
        }
    }
}

abstract class User {
    protected ChatMediator mediator;
    protected String name;

    public User(ChatMediator mediator, String name) {
        this.mediator = mediator;
        this.name = name;
    }

    public abstract void send(String message);

    public abstract void receive(String message);
}

class ChatUser extends User {
    public ChatUser(ChatMediator mediator, String name) {
        super(mediator, name);
    }

    public void send(String message) {
        System.out.println(this.name + " sends: " + message);
        mediator.sendMessage(message, this);
    }

    public void receive(String message) {
        System.out.println(this.name + " receives: " + message);
    }
}

public class Example1InterpreterClient {
    public static void main(String[] args) {
        ChatMediator chatRoom = new ChatRoom();
    }
}
```

```

        User user1 = new ChatUser(chatRoom, "Alice");
        User user2 = new ChatUser(chatRoom, "Bob");
        User user3 = new ChatUser(chatRoom, "Charlie");

        chatRoom.addUser(user1);
        chatRoom.addUser(user2);
        chatRoom.addUser(user3);

        user1.send("Hello everyone!");
    }
}

```

Output:

```

Alice sends: Hello everyone!
Bob receives: Hello everyone!
Charlie receives: Hello everyone!

```

Example2: Air Traffic Control

```

package com.mannemlokes.example2;

import java.util.ArrayList;
import java.util.List;

interface ATCMediator {
    void registerFlight(Flight flight);

    void notifyOthers(Flight origin);
}

class ATC implements ATCMediator {
    private List<Flight> flights = new ArrayList<>();

    public void registerFlight(Flight flight) {
        flights.add(flight);
    }

    public void notifyOthers(Flight origin) {
        for (Flight flight : flights) {
            if (flight != origin) {
                flight.receiveSignal();
            }
        }
    }
}

abstract class Flight {
    protected ATCMediator atc;

    public Flight(ATCMediator atc) {
        this.atc = atc;
    }

    public abstract void sendSignal();

    public abstract void receiveSignal();
}

```

```

class CommercialFlight extends Flight {
    private String flightCode;

    public CommercialFlight(ATCMediator atc, String flightCode) {
        super(atc);
        this.flightCode = flightCode;
    }

    public void sendSignal() {
        System.out.println(flightCode + " sending signal.");
        atc.notifyOthers(this);
    }

    public void receiveSignal() {
        System.out.println(flightCode + " received another flight signal.");
    }
}

public class Example2ATCClient {
    public static void main(String[] args) {
        ATCMediator atc = new ATC();

        Flight flight1 = new CommercialFlight(atc, "AI202");
        Flight flight2 = new CommercialFlight(atc, "BA303");

        atc.registerFlight(flight1);
        atc.registerFlight(flight2);

        flight1.sendSignal();
    }
}

```

Output:

```

AI202 sending signal.
BA303 received another flight signal.

```

Example3: UI Dialog Component Mediator

```

package com.mannemlokes.example3;

interface DialogMediator {
    void notify(Component sender, String event);
}

abstract class Component {
    protected DialogMediator mediator;

    public Component(DialogMediator mediator) {
        this.mediator = mediator;
    }

    public abstract void trigger(String event);
}

class Button extends Component {
    public Button(DialogMediator mediator) {

```

```

        super(mediator);
    }

    public void trigger(String event) {
        System.out.println("Button clicked");
        mediator.notify(this, event);
    }
}

class TextBox extends Component {
    public TextBox(DialogMediator mediator) {
        super(mediator);
    }

    public void setText(String text) {
        System.out.println("TextBox updated: " + text);
    }

    public void trigger(String event) {
        // not used
    }
}

class Dialog implements DialogMediator {
    private Button button;
    private TextBox textBox;

    public void registerComponents(Button button, TextBox textBox) {
        this.button = button;
        this.textBox = textBox;
    }

    public void notify(Component sender, String event) {
        if (sender == button && "click".equals(event)) {
            textBox.setText("Form Submitted");
        }
    }
}

public class Example3DialogClient {
    public static void main(String[] args) {
        Dialog dialog = new Dialog();
        Button button = new Button(dialog);
        TextBox textBox = new TextBox(dialog);

        dialog.registerComponents(button, textBox);
        button.trigger("click");
    }
}

```

Output:

```

Button clicked
TextBox updated: Form Submitted

```

19.9 Mediator vs Other Patterns

Pattern	Purpose
---------	---------

Mediator	Centralize communication
Observer	Notify subscribers
Facade	Simplify subsystem interface

19.10 Pros and Cons



Pros

- Reduces coupling
- Makes system easier to maintain
- Centralized logic



Cons

- Mediator can become complex if overused
- May become a god object if not designed properly

19.11 When to Use / Not to Use

Use When:

- Many classes are tightly coupled
- You want to simplify communication logic

Avoid When:

- Simple interactions (overhead not worth it)

19.12 Summary

Feature	Description
Intent	Centralize object communication
Use Case	UI events, chatroom, ATC
Key Benefit	Loose coupling and maintainable code

The **Mediator Pattern** centralizes complex object interactions, improves maintainability, and promotes cleaner architecture.